



INF8770 : Technologies Multimédias

TP1 : Compression sans perte : texte

Automne 2026

Étudiants :

Capistran-Champoux, Charlotte (Matricule : 2353263)

Croteau, Alexis (Matricule : 2205235)

Chargé de laboratoire :

Nom du Chargé de Laboratoire

Département de génie informatique et génie logiciel
Polytechnique Montréal

Date de remise : 28 septembre 2026 à 17h00 sur Moodle

Table des matières

1	Introduction	2
2	Réponses aux Questions	2
2.1	Question 1 : Métriques d'analyse et choix des textes	2
2.2	Question 2 : Choix de la méthode, argumentation et attentes	4
2.3	Question 3	5
2.4	Question 4 : [Titre de la Question 4]	5
3	Conclusion	5
A	Annexe : Données complémentaires et extraits de code	6

1 Introduction

Cette section présente l'objectif du travail pratique et le contexte des travaux réalisés. Expliquez brièvement la problématique traitée, les concepts théoriques abordés ainsi que les technologies ou données utilisées dans le cadre de vos expérimentations.

Rappel : Ce gabarit est limité à un maximum strict de 10 pages, annexes comprises. Veillez à rester synthétique, rigoureux et précis dans vos explications.

Ce travail a pour objectif l'expérimentation avec les méthodes de compression sans perte vues en classe, dans le but d'en obtenir une compréhension plus pratique.

2 Réponses aux Questions

2.1 Question 1 : Métriques d'analyse et choix des textes

Les textes choisis sont le texte littéraire en français, les données structurées météo et le code source en Python.

Présentation des métriques

Les différentes métriques présentées dans le Tableau 1 permettront une analyse plus exhaustive de chaque fichier, en exposant certaines valeurs clés utiles. Dans un premier temps, le tableau présente la taille du fichier en octets, ainsi que le nombre de caractères qu'il contient. Puisque très souvent, un caractère est codé par un seul octet, ces deux valeurs devraient s'approximer. Ensuite, le tableau expose la taille de l'alphabet, soit la longueur de la liste de tous les symboles différents que contient le fichier, encore une fois exprimée en octets, et en nombre de symboles. Ces deux valeurs seront à nouveau à peu près équivalentes, pour la même raison exprimée plus tôt.

La métrique suivante présente l'entropie, soit le nombre minimal de bits nécessaires, en moyenne, pour exprimer un symbole du fichier à l'étude, en employant une méthode traitant chaque symbole individuellement. Elle est calculée en prenant en compte la probabilité d'occurrence de chaque symbole dans le texte, et est un indicateur de l'équilibre d'apparition entre ceux-ci. Un fichier présentant une forte entropie aura donc une occurrence assez équivalente d'apparition de chacun des symboles de son alphabet, alors qu'un fichier plus déséquilibré aura une entropie plus faible. Cette valeur minimale ne borne par contre que les modèles d'ordre 0, tel le modèle de Huffman, et un code qui prend en compte les séquences de symboles pourrait la surpasser. Le ratio entropie sur entropie maximale représente simplement l'économie que génère un modèle d'ordre 0 optimal par rapport à un fichier encodé sans compression. Découlant directement de l'entropie, la taille théorique minimale multiplie cette valeur de bits par la taille du fichier, nous proposant le meilleur scénario de taille que pourrait produire une compression avec un modèle traitant les symboles individuellement.

Finalement, le ratio de mots uniques découle d'une manipulation un peu plus complexe, et est un indicateur approximatif de la performance attendue d'un algorithme exploitant les séquences. Cette valeur implique d'abord la séparation du fichier en mots, soit des séries de caractères séparées par des délimiteurs. Dans le but de produire un résultat optimal tout en gardant une règle uniforme pour les trois fichiers, le choix des caractères a été réalisé suite à une analyse des fichiers et de leurs structures répétitives. Le ratio de mots uniques nous indique ainsi la propension de répétition des mêmes mots dans le texte. Un ratio élevé suppose un texte composé de peu de répétitions, alors qu'un ratio faible signifie qu'un petit nombre de mots différents se répèteront souvent.

TABLE 1 – Mesures quantitatives des trois fichiers analysés.

Métrique	Littéraire (fr)	Structuré (CSV)	Code (Python)
Taille du fichier [octets]	5092	26515	3844
Nombre de symboles [caractères]	4958	26515	3841
Taille de l'alphabet [octets]	62	55	68
Taille de l'alphabet [caractères]	60	55	67
Entropie $H(X)$ [bits/octet]	4.42	4.97	4.37
Entropie $H(X)$ [bits/caractère]	4.36	4.97	4.36
Entropie relative $H(X)/H_{max}$ [%]	74.26	85.94	71.75
Taille théorique minimale [octets]	2814.29	16467.90	2098.62
Ratio mots uniques / mots totaux [%]	53.16	6.20	39.56

Analyse du texte littéraire en Français.

Dabord, le texte littéraire en français présente une métrique se distinguant, soit la différence assez marquée entre la taille du fichier et en octets et en nombre de caractères. Celle-ci s'explique par la méthode d'encodage UTF-8, qui emploie deux octets pour exprimer les caractères accentués : un premier symbole est universel à tous les caractères accentués, alors que le second distingue parmi ceux-ci. Ainsi, la différence nous indique que ce texte sera composée d'une quantité importante de caractères accentués.

De par sa nature littéraire, le fichier sera certainement composé de certains symboles se répétant très souvent, par exemple l'espace. L'entropie relative d'environ 74% renforce cette intuition de relatif déséquilibre dans la répartition des caractères, ce qui favorisera l'encodage Huffman. Dans un autre ordre d'idée, le texte littéraire est le plus diversifié des trois en termes de séquences répétitives, présentant un ratio de 53% de mots uniques. Ainsi, un algorithme basé sur les structures répétitives pourrait générer un résultat peu satisfaisant.

Analyse des données structurées météo

D'autre part, les données structurées météo présentent une entropie relative élevée. Cette métriques nous indique que ce texte n'est probablement pas très adapté à un encodage par Huffman. En effet, la répartition assez régulière de caractères diversifiés ne permettra pas d'exploiter l'avantage principal de Huffman, soit l'encodage de caractères fréquents avec très peu de bits.

De plus, on constate que ce texte présente une très importantes répétition des mêmes structures, indiquée par le ratio de mots uniques très faible. Il est donc un candidat idéal pour une méthode d'encodage considérant les séquences répétitives, qui produira un résultat nettement plus satisfaisant que Huffman.

Analyse du code source en Python

Finalement, le code source en Python relève l'entropie relative la plus faible, nous indiquant un fort déséquilibre de répartition entre les différents symboles de son alphabet. Ceci est attendu, puisqu'un fichier de code est composé de caractères à forte répartition de par leur emploi dans le contexte, par exemple les parenthèses, espaces de l'indentation ou autre marqueurs de structure. Ainsi, on pourra s'attendre à un résultat satisfaisant par l'algorithme de Huffman, qui saura exploiter ce déséquilibre.

Par contre avec un ratio de mots uniques relativement faible, il est probable qu'un algorithme employant une méthode prenant en compte les structures répétitives sera également efficace. Les langages de code

sont composés de mots qui se répètent souvent, et qui pourront être encodés plus efficacement s'ils sont considérés dans leur ensemble.

En bref, l'encodage de Huffman génère une économie d'espace importante dans le cas de textes présentant une entropie relative faible. En effet, il exploite ainsi le déséquilibre de répartition en encodant les caractères fréquents avec peu de bits. Par contre, un texte composé d'une répétition fréquente de quelques chaînes de caractère bénéficiera plutôt d'un algorithme sachant exploiter cette forme particulière.

2.2 Question 2 : Choix de la méthode, argumentation et attentes

L'analyse des trois fichiers à l'étude nous indique un fait clair : la redondance s'y trouve plutôt dans les séquences que dans les symboles. En effet, bien que le ratio de mots uniques varie grandement, il n'excède pas environ 53%, indiquant que près d'un mot sur deux est répété. Puisque le modèle de Huffman ne considère pas ces répétitions dans son application, il faut explorer d'autres méthodes qui sauront produire un résultat plus satisfaisant. La méthode choisie doit être sans perte et s'appliquer aux trois types de fichiers tel que spécifié en énoncé. Elle doit aussi bien entendu exploiter les séquences, et rester raisonnable à implémenter.

Exploration des méthodes vues en classe

Parmi les méthodes vues en classe qui s'appliquent aux documents textuels, on compte l'encodage arithmétique. Celui-ci est encore un modèle d'ordre 0, et n'élimine que les arrondis de Huffman à des bits entiers. Son gain serait donc marginal.

Le codage prédictif ou de Golomb-Rice est plutôt conçu pour des signaux numériques où les valeurs voisines sont corrélées, et peu pour du texte. En effet, la valeur numérique d'un octet de texte n'a pas de lien avec celle de son voisin, et encoder un lien entre elles serait peu pertinent.

Parmi les méthodes plus prometteuses, on compte l'encodage par paires (BPE). Cet algorithme prend bien en compte les séquences répétées, en remplaçant itérativement les paires fréquentes par un octet libre. Puisque les alphabets à l'étude ne comptent qu'entre 55 et 68 caractères parmi les 256 valeurs possibles, il resterait amplement de valeurs libres que cet algorithme pourrait exploiter. Par contre, l'encodage par paires nécessite un dictionnaire imposant, qu'il faut encoder à même l'en-tête du fichier. Il devient donc pertinent dans le cas de fichiers de grande taille et peu pour des fichiers plus réduits comme ceux étudiés ici.

L'encodage LZ77 est également un candidat à évaluer. Son utilisation d'une fenêtre glissante capturerait bien les répétitions de séquences dans le fichier CSV ou de code. Par contre, l'emploi de triplets a un coût fixe de bits, même si la correspondance est très courte ou absente. De plus, il est assez complexe à implémenter, pour un avantage assez incertain sur les fichiers à l'étude, surtout le fichier littéraire composé de moins de répétitions.

Choix de la méthode et argumentation

La méthode retenue pour l'analyse des fichiers dans le cadre de ce travail pratique est l'encodage LZW. Cet algorithme conserve l'avantage des méthodes par dictionnaire, puisqu'il exploite bien les séquences. Il n'émet qu'un seul code par séquence, mais surtout, son dictionnaire n'est pas transmis en en-tête du fichier. Ainsi, il ne présente pas ce coût fixe imposé par d'autres méthodes et non négligeable sur de petits fichiers, puisque son dictionnaire peut être reconstruit directement par l'algorithme de décodage.

On peut donc s'attendre à une économie substantielle d'espace dans le cas du fichier CSV, dont l'entropie relative est très haute, pénalisant Huffman, et le ratio de mots uniques très bas, favorisant LZW.

Le fichier Python risque de générer une économie plus modeste, mais tout de même conséquente étant donné la répétition de séquences qu'on y retrouve. Moins définitif, le fichier littéraire présente moins de structures répétées, ainsi le gain d'espace pourrait être plus réduit. Par contre, le codage LZW y propose toujours l'avantage de ne pas avoir à stocker un dictionnaire en en-tête, contrairement à Huffman.

2.3 Question 3

Pour ce qui est de l'implémentation logicielle des algorithmes de Huffman et LZW, nous avons fait le choix d'écrire le code nous-même. Cette décision a été prise puisqu'on voulait se familiariser avec ces concepts vus en cours, plutôt que d'utiliser une librairie qui implémente ces algorithmes pour nous. Nous avons choisi de les implémenter en Python, puisque c'est un langage avec lequel nous sommes familiers. Notre référence pour savoir comment implémenter ces deux algorithmes correctement était les notes de cours, qui décrivent très clairement le fonctionnement des algorithmes.

Pour valider que nos implémentations fonctionnent correctement sans pertes, nous avons comparé les textes originaux aux textes décodés. Nous avons donc constaté qu'ils étaient identiques, ce qui nous confirme que l'implémentation des deux algorithmes est bel et bien sans perte.

Pour comparer les performances des algorithmes, nous comparons simplement la taille des textes originaux avec la taille de leur version comprimée, cela nous permet de déterminer quel algorithme est le plus efficace entre Huffman et LZW.

2.4 Question 4 : [Titre de la Question 4]

Présentez vos résultats expérimentaux, en intégrant des tableaux ou des graphiques pour illustrer vos observations et valider vos hypothèses initiales. Discutez des performances obtenues et analysez les limites de votre approche.

La Figure 1 donne un exemple de graphique comparatif. N'oubliez pas d'inclure une légende descriptive et les unités sur les axes.

3 Conclusion

Résumez ici les principaux constats de votre travail pratique, la conformité de vos résultats face aux objectifs initiaux et les recommandations d'ingénierie qui en découlent.

Références

- [1] A. Auteur, « Titre de l'article ou de la publication », *Nom de la Revue ou Conférence*, vol. 10, no. 3, pp. 120–130, 2023.
- [2] B. Smith et C. Jones, *Titre du manuel de référence ou de la documentation*, 3e éd., Éditeur, 2024.

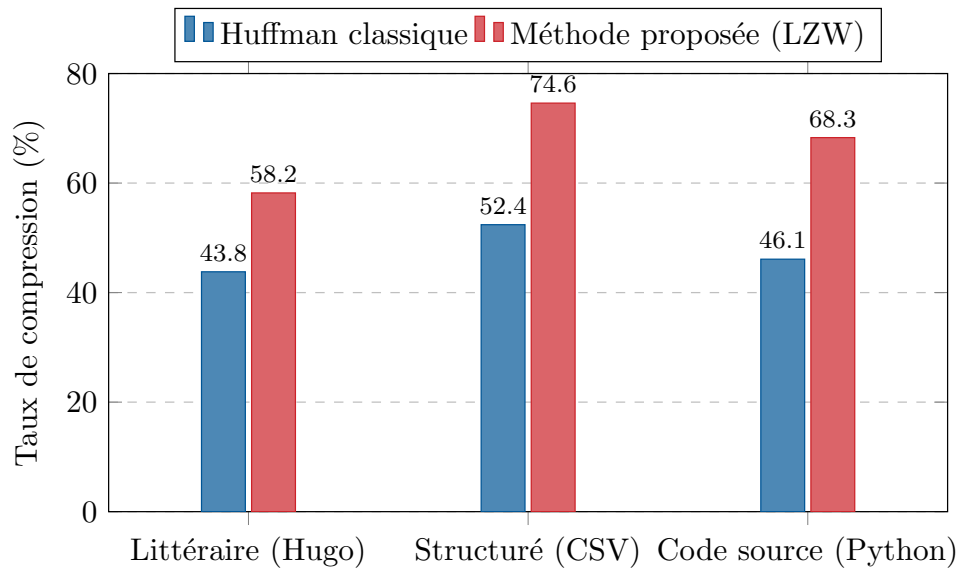


FIGURE 1 – Courbe ou histogramme comparatif des performances obtenues selon la méthode testée. Veuillez à expliciter la légende et les unités employées.

A Annexe : Données complémentaires et extraits de code

Cette annexe contient les données supplémentaires (calculs détaillés, tableaux de distribution étendus) ainsi que les extraits de code source clés développés pour le travail pratique.

```

1 # Exemple d'extrait de code developpe par les etudiants
2 def traitement_multimedia(fichier_entree: str, fichier_sortie: str):
3     """
4     Fonction principale de traitement ou de compression des donnees.
5     """
6     # Insérer ici les instructions clés de votre algorithme
7     print(f"Traitement de {fichier_entree} vers {fichier_sortie}")

```

Listing 1 – Exemple d'extrait de code source pour le traitement multimédia.